

复现 Grounding DINO: 开放词汇目标检测与视觉定位

Reproducing Grounding DINO: Open-Vocabulary Object
Detection
and Visual Grounding on COCO 2017

苏思齐 (12310645) 杨官宇涵 (12313614) 莫丰源 (12311805)

南方科技大学 (SUSTech), 深圳

2026 年 6 月 21 日



南方科技大学
SOUTHERN UNIVERSITY OF SCIENCE AND TECHNOLOGY

Introduction

Related Works

Method

Experiments

4.1 Datasets

4.2 Implementation Details

4.3 Metrics

4.4 Experimental Design & Results

4.5 Ablation Experiments

Conclusion

References

Contributions

1. Introduction

Task Background

- ▶ **Open-vocabulary object detection** localizes objects described by arbitrary text, rather than a fixed training label set.
- ▶ **Visual grounding** further connects natural-language expressions to image regions, e.g., “the red car on the left”.
- ▶ The task combines object recognition, localization, and language understanding in one pipeline.

Main Challenges

- ▶ Aligning visual regions with free-form text prompts
- ▶ Generalizing to categories not explicitly fixed in the detector head
- ▶ Balancing recall and precision through prompt and threshold design

Our Project

- ▶ Reproduce a Grounding DINO detection pipeline
- ▶ Evaluate on COCO-style detection metrics
- ▶ Analyze prompt format and threshold behavior
- ▶ Build a demo for image upload, text prompt input, and visualized detections

0.559

Best mAP

0.735

Best AP50

5

Best Epoch

0.712

AP_l

Best recorded checkpoint in the 6-epoch run: epoch 5.

2. Related Works

Detection Transformers

- ▶ **DETR** formulates object detection as end-to-end set prediction with transformers.
- ▶ **Deformable DETR** improves convergence by attending to sparse multi-scale feature locations.
- ▶ **DINO** further strengthens DETR-style detection with denoising training and better query initialization.

Vision-Language Pre-training

- ▶ **CLIP** learns transferable image-text representations from large-scale natural-language supervision.
- ▶ **BERT** provides contextual token embeddings for phrase-level language understanding.

Open-Vocabulary Detection and Grounding

- ▶ **GLIP** unifies object detection and phrase grounding through grounded language-image pre-training.
- ▶ **OWL-ViT** adapts vision transformers for open-vocabulary localization with text queries.
- ▶ **YOLO-World** targets real-time open-vocabulary detection with efficient prompt-aware prediction.

Why Grounding DINO

Grounding DINO marries DINO-style detection with grounded pre-training, using a Swin visual backbone, a BERT text encoder, and cross-modal fusion. It directly supports text-prompted boxes, which matches our reproduction and demo goals.

3.1 Model Architecture

Three Core Components

1. Swin Transformer (Liu et al., 2021)

- 4-stage hierarchical features (stride 4/8/16/32)
- Shifted-window attention, 28M params

2. BERT Encoder (Devlin et al., 2019)

- bert-base-uncased: 12 layers, hidden=768
- Contextualized token embeddings ($L \leq 256$)

3. Deformable Fusion

- 6 layers, 8 heads, 900 object queries
- Cross-modal: visual locations $\leftrightarrow$ text tokens

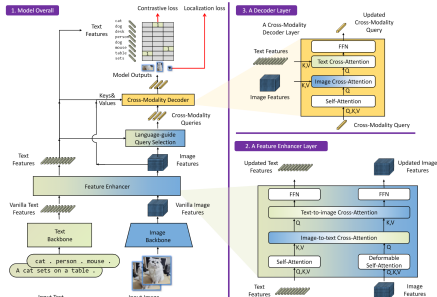
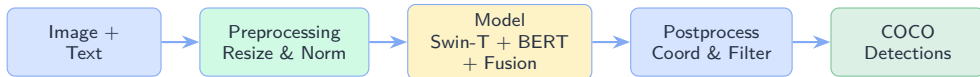


Figure 3. The framework of Grounding DINO. We present the overall framework, a feature enhancer layer, and a decoder layer in block 1, block 2, and block 3, respectively.

Reproduction: zero-shot, fixed thresholds, single-scale, no fine-tuning.

3.2 Inference Pipeline



Key Parameters

- ▶ Input: short side 800px, long side ≤ 1333 px
- ▶ Normalization: ImageNet mean/std
- ▶ Thresholds: box_th=0.35, text_th=0.25
- ▶ 900 queries $\times$ 256-dim features

Speed

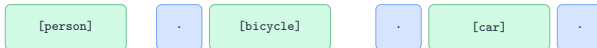
- ▶ ~ 0.81 s/image on RTX 4060
- ▶ ~ 68 min for full COCO val2017 (5,000 images)
- ▶ GPU memory peak: ~ 3.2 GB

3.3 Prompt Design: Why Dot-Separated?

BERT WordPiece Tokenizer —how the prompt is split into tokens:

✓ **Dot-separated:**

"person . bicycle . car ."



✗ **Comma-separated:**

"person, bicycle, car,"



Why this matters

- ▶ Period surrounded by spaces → independent separator token
- ▶ Comma merges with preceding word → ambiguous boundaries
- ▶ Non-dot formats cause **~62% AP drop**

Phrase → Category

1. **Exact match** (case-insensitive)
 2. **Substring match** ("dining" → "dining table")
 3. **Alias match** (21-entry dict)
- ~2–5% phrases discarded when all layers fail.

3.4 Software Architecture

5 Core Modules (1,826 LOC)

Module	Role	LOC
models/	Model wrapper	182
inference/predictor.py	Inference API	229
inference/visualizer.py	Draw boxes	262
engine/evaluator.py	COCOeval	368
datasets/coco_categories.py	Prompt & map	269
datasets/coco_dataset.py	Data utils	188
utils/box_ops.py	IoU, NMS	115

Design

- ▶ **Config-driven:** OmegaConf YAML, CLI overrides
- ▶ **Modular:** 5 modules + 10 CLI scripts
- ▶ **Error-tolerant:** per-image try/except

CLI Scripts (10)

inference.py | eval.py | run_experiments.py
download_weights.py | download_coco.py
generate_report_highlights.py | generate_visualizations.py
train.py | setup_env.sh | setup_env.ps1

4.1 COCO 2017 Validation Set

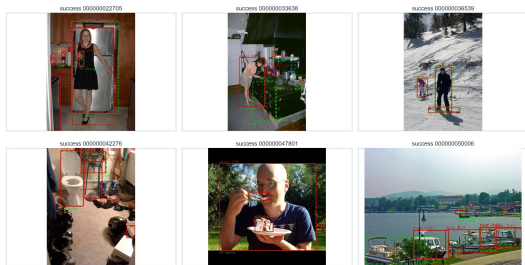
Overview (Lin et al., ECCV 2014)

- ▶ 5,000 RGB images, diverse real-world scenes
- ▶ 80 categories (person, car, dog, chair, ...)
- ▶ 36,781 GT boxes —avg ~ 7.4 per image
- ▶ Long-tail distribution: “person” dominates; “hair drier” < 100

Splits

Split	Images	Usage
train2017	118K	Fine-tuning
val2017	5,000	Evaluation
test2017	40K	Benchmark

Qualitative success cases



Example COCO val2017 images with Grounding DINO zero-shot detections

4.2 Implementation Details

- ▶ **Model:** Grounding DINO with Swin-T backbone.
- ▶ **Framework:** PyTorch + MMDetection + MMEEngine.
- ▶ **Initialization:** official Grounding DINO Swin-T checkpoint.
- ▶ **Language model:** local bert-base-uncased, used offline to avoid remote Hugging Face dependency.
- ▶ **Dataset format:** COCO detection format.
 - Training set: train2017
 - Validation set: val2017
 - Annotation files: instances_train2017.json, instances_val2017.json
- ▶ **Training strategy:** fine-tune the pretrained model for 6 epochs and save one checkpoint after each epoch.

4.2 Implementation Details: Hyper-parameters

Item	Setting
Backbone	Swin-T
Language model	BERT-base-uncased
Optimizer	AdamW
Learning rate	5×10^{-5}
Batch size	2
Mixed precision	AMP enabled
Training epochs	6
Checkpoint interval	1 epoch
Validation interval	1 epoch
GPU	NVIDIA RTX 4090

For inference, a demo script is implemented to support both single-image and batch-image prediction. It takes an image and a text prompt as input, then saves both annotated images and JSON prediction results.

4.2 Implementation Details: MMDetection Inference

- ▶ **Class-aware prompt encoding:** the COCO class names are used as text entities, such as person . car . dog .
- ▶ **Token-positive map:** MMDetection records which BERT tokens correspond to each category. For example, the class traffic light may correspond to the tokens traffic and light.
- ▶ **Score conversion:** Grounding DINO predicts box-query scores over text tokens. MMDetection uses the token-positive map to convert token-level scores into class-level scores.
- ▶ **Top-K selection:** instead of applying a fixed box threshold before evaluation, MMDetection selects the top `max_per_img=300` query-class predictions.
- ▶ **Difference from the original script:** the original script filters boxes using a fixed `box_threshold`, which may remove low-score candidates before COCO evaluation and reduce recall.
- ▶ **Demo threshold:** in our frontend demo, `box_threshold` is applied after MMDetection top-K inference to hide low-confidence boxes for visualization.
- ▶ **Text threshold:** in the MMDetection COCO-style path, categories are mainly determined by the token-positive map. Therefore, `text_threshold` no longer directly controls the number of boxes, but is kept for interface compatibility and possible open-vocabulary phrase matching.

We use the standard COCO object detection metrics to evaluate bounding box detection performance.

- ▶ **bbox mAP**: mean Average Precision averaged over IoU thresholds from 0.50 to 0.95. This is the main overall metric.
- ▶ **bbox mAP@50**: AP at IoU threshold 0.50. It reflects whether the model can roughly locate objects.
- ▶ **bbox mAP@75**: AP at IoU threshold 0.75. It requires more accurate bounding box localization.
- ▶ **bbox mAP_s**, **bbox mAP_m**, **bbox mAP_l**: AP for small, medium, and large objects respectively.

The final demo checkpoint is selected mainly based on overall **bbox mAP**, while also considering mAP@50, mAP@75, and object-scale-specific AP.

4.4 Experimental Design

- ▶ First, we evaluate the official pretrained Grounding DINO Swin-T checkpoint as the baseline.
- ▶ Then, we fine-tune the model on the COCO training set.
- ▶ A checkpoint is saved after each epoch from epoch 1 to epoch 6.
- ▶ Each checkpoint is evaluated on the COCO validation set.
- ▶ The goal is to observe whether fine-tuning improves detection performance and to select the best checkpoint for the final demo.

This design allows us to compare the pretrained model with fine-tuned checkpoints and analyze whether longer training continues to improve the model.

4.4 Experimental Results

Model	mAP	AP50	AP75	AP _s	AP _m	AP _l
Baseline	0.485	0.645	0.530	0.342	0.518	0.635
Epoch 1	0.546	0.718	0.600	0.379	0.583	0.693
Epoch 2	0.553	0.722	0.606	0.400	0.592	0.701
Epoch 3	0.556	0.726	0.610	0.393	0.594	0.703
Epoch 4	0.555	0.729	0.611	0.394	0.595	0.703
Epoch 5	0.559	0.735	0.615	0.400	0.593	0.712
Epoch 6	0.558	0.733	0.614	0.394	0.590	0.704

Epoch 5 achieves the best overall performance and is selected as the final checkpoint for the demo.

4.4 Result Analysis

- ▶ Fine-tuning significantly improves the model compared with the pretrained baseline.
- ▶ Overall bbox mAP improves from **0.485** to **0.559**, with an absolute gain of **0.074**.
- ▶ AP50 improves from **0.645** to **0.735**, showing better object recognition and coarse localization.
- ▶ AP75 improves from **0.530** to **0.615**, indicating more accurate bounding box localization.
- ▶ Epoch 6 does not further improve over epoch 5, suggesting the model has nearly converged after 5 epochs.

Therefore, we use **epoch 5** as the final demo model because it has the highest mAP, AP50, AP75, and large-object AP.

4.4 Sample Visualization

The demo system takes an image and a text prompt as input, then outputs detected objects with bounding boxes, confidence scores, and matched phrases.

- ▶ Input: image from the demo set.
- ▶ Text prompt example: person . car . dog .
- ▶ Output: annotated image and JSON prediction file.

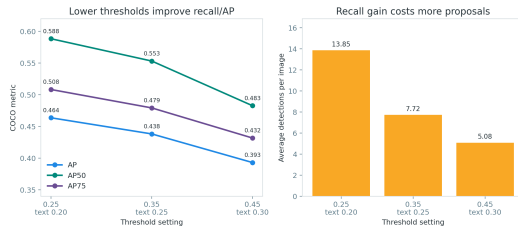
4.5 Ablation: Threshold Sensitivity

Experiment Design

- ▶ Three threshold combos tested on COCO 500-image subset
- ▶ **Box th**: min confidence for detected boxes
- ▶ **Text th**: min confidence for text-token match
- ▶ Fixed model: Grounding DINO Swin-T, zero-shot

Results (subset-500)

Run	box_th	text_th	AP	AP50	Box/Img
A1	0.25	0.20	0.4637	0.5884	13.85
A2	0.35	0.25	0.4382	0.5532	7.72
A3	0.45	0.30	0.3931	0.4827	5.08



Key Insight

- ▶ Lower threshold → **higher AP** but **3× more boxes**
- ▶ AP drops **15%** from A1 to A3 (box_th 0.25 → 0.45)
- ▶ Trade-off: recall vs. precision for downstream tasks

4.5 Ablation: Prompt Format Comparison

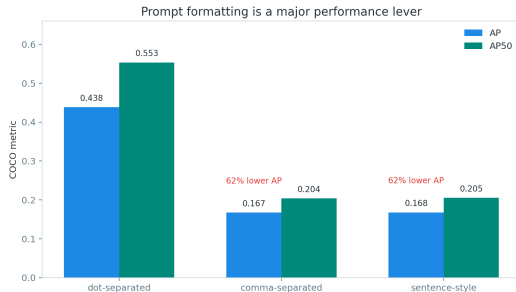
Experiment Design

- ▶ Three prompt formats on 500-image subset
- ▶ Same thresholds: box_th=0.35, text_th=0.25
- ▶ All 80 COCO categories included in each prompt

Results

Format	AP	AP50
dot-separated	0.4382	0.5532
comma-separated	0.1671	0.2038
sentence-style	0.1675	0.2051

Non-dot formats cause $\sim 62\%$ AP drop



Why Dot-Separated Wins

- ▶ BERT WordPiece splits ``.'`' as **independent token**
- ▶ Comma merges with preceding word $\rightarrow$ ambiguous boundaries
- ▶ Sentence format adds noise tokens that dilute attention

5. Conclusion

What We Did

- ▶ Reproduced Grounding DINO (Swin-T) with official weights.
- ▶ Evaluated zero-shot on full COCO val2017 (5,000 images).
- ▶ Built config-driven pipeline (5 modules, 10 scripts).

Challenge

- ▶ COCO uses fixed category IDs; model outputs free-form phrases.
- ▶ Phrase-to-category mapping required for evaluation.
- ▶ Zero-shot AP still lags behind supervised detectors.

Key Findings

1. Dot-separated prompts are essential (others drop AP 62%).
2. Lower thresholds improve AP but add $3\times$ detections.
3. Small objects ($APS=0.26$) are $2.1\times$ worse than large ($APL=0.55$).

Future Work

- ▶ Fine-tune on COCO train for better performance.
- ▶ Cross-dataset evaluation (ODinW, LVIS).
- ▶ Compare with YOLO-World, OWL-ViT under same protocol.

1. **Grounding DINO**: S. Liu et al., “Grounding DINO: Marrying DINO with Grounded Pre-Training for Open-Set Object Detection,” *ECCV*, 2024.
2. **DINO**: H. Zhang et al., “DINO: DETR with Improved DeNoising Anchor Boxes for End-to-End Object Detection,” *ICLR*, 2023.
3. **DETR**: N. Carion et al., “End-to-End Object Detection with Transformers,” *ECCV*, 2020.
4. **Deformable DETR**: X. Zhu et al., “Deformable DETR: Deformable Transformers for End-to-End Object Detection,” *ICLR*, 2021.
5. **Swin Transformer**: Z. Liu et al., “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows,” *ICCV*, 2021.
6. **BERT**: J. Devlin et al., “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL*, 2019.
7. **GLIP**: L. H. Li et al., “Grounded Language-Image Pre-training,” *CVPR*, 2022.
8. **OWL-ViT**: M. Minderer et al., “Simple Open-Vocabulary Object Detection with Vision Transformers,” *ECCV*, 2022.
9. **YOLO-World**: T. Cheng et al., “YOLO-World: Real-Time Open-Vocabulary Object Detection,” *CVPR*, 2024.
10. **COCO**: T.-Y. Lin et al., “Microsoft COCO: Common Objects in Context,” *ECCV*, 2014.
11. **CLIP**: A. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision,” *ICML*, 2021.

苏思齐 (12310645)

- ▶ 设计和执行微调方案
- ▶ 部分 ppt
- ▶ 参与修改后端推理脚本

杨官宇涵 (12313614)

- ▶ 整体项目设计与实现
- ▶ 全部 Python 模块与 CLI 脚本编写
- ▶ 完整 COCO 评估与消融实验执行
- ▶ 报告撰写

莫丰源 (12311805)

- ▶ 参与前期参数初调, 包括 box/text threshold 与 prompt 格式
- ▶ 设计 demo 前端页面, 展示训练指标与实时检测结果
- ▶ 完成前后端落地对接, 支持图片上传、文本输入和结果可视化

Thank You! Q & A